

# ChatGPT Plus / Pro × Chat / Work / Codex

## 数学建模与大学科研使用指南

基于本轮问答提炼 · 复盘版

### 一句话总纲

Chat 负责“想清楚”，Work 负责“把复杂任务组织并交付”，Codex 负责“真正写代码、运行、调试和维护工程”。Plus/Pro 不需要本地接 API 才能使用；网页端和官方 App 就能享受订阅权益。

适合：数学建模国赛备赛 · 课程项目 · 进实验室前准备 · 科研入门

资料校验日期：2026-09-04

## 1. 先记住这 8 个结论

- Plus / Pro 是 ChatGPT 订阅，不要求你在本地接 OpenAI API。网页端、官方 App 等直接登录即可使用对应权益。
- ChatGPT 订阅与 API 是两套独立计费体系；买 Plus / Pro 不等于 API 免费。
- 只做轻量聊天时，Plus 的价值感不会特别夸张；当你开始大量使用文件分析、Deep Research、Work、Codex 时，价值会明显放大。
- Pro 不是“回答质量突然强十倍”，它更像是面向高强度复杂任务：更高档模型/能力、更高 agentic 使用额度，更适合长时间 Work / Codex 会话。
- 数学建模里最推荐的分工：Chat 决策模型路线，Codex 执行代码与实验，Work 组织资料与最终成果。
- AI 不应替你决定“为什么选这个模型”；模型选择、假设合理性、结果含义必须保留在人手里。
- 科研准备的核心不是“会问 AI”，而是建立可重复的工作流：文献 → 假设 → 代码 → 数据 → 实验 → 图表 → 写作 → 复现。
- 对大学生而言：平时 Plus 通常够用；只有在数模/科研高强度阶段频繁撞 Work/Codex 限额时，再考虑临时升级 Pro。

### 你的最佳升级判断

不要问“Pro 强不强”，而要问“Plus 是否已经限制了我的工作流”。如果你一周的 Codex / Work 都用不满，升级 Pro 的边际收益通常不高。

## 2. Plus、Pro、API：到底是什么关系？

| 项目       | Plus                         | Pro                  | API                 |
|----------|------------------------------|----------------------|---------------------|
| 主要用途     | 日常学习、研究、文件分析、Chat/Work/Codex | 高强度复杂任务、长时间 agent 工作 | 把模型接入自己的程序、网站、自动化系统 |
| 是否必须本地配置 | 不需要                          | 不需要                  | 通常需要开发配置            |
| 网页端可直接用  | 可以                           | 可以                   | 不是同一产品入口            |
| 核心区别     | 额度与高级功能                      | 更高强度能力与用量            | 按 API 使用单独计费        |
| 适合你现在吗   | 是，优先                         | 比赛/科研冲刺期再评估          | 有自动化/开发需求再学         |

当前官方信息：Plus 为 \$20/月；Pro 当前提供更高强度档位，计划与额度可能调整，最终以 ChatGPT 账户内 Pricing / Billing 页面为准。

### 3. Chat、Work、Codex：一张表看懂分工

| 工具    | 最擅长的问题              | 数学建模中的角色                    | 科研中的角色                 |
|-------|---------------------|-----------------------------|------------------------|
| Chat  | “为什么？用什么？合理吗？”      | 读题、拆题、选模型、解释结果、审稿           | 理解论文、讨论假设、设计实验、解释结果    |
| Work  | “如何把很多材料串成一个完整交付物？” | 任务计划、跨文件整理、论文/报告初稿          | 文献/实验材料整合、项目周报、研究报告    |
| Codex | “怎么真正把它实现并跑起来？”     | 写 Python/MATLAB、调试、批量实验、可视化 | 科研代码库、数据处理、参数扫描、复现与自动化 |

记忆口诀

Chat = 决策；Work = 统筹与交付；Codex = 执行。三者不是一条单向流水线，而是不断循环：Codex 产出事实 → Chat 解释 → 决策下一步 → Codex 执行 → Work 整理。

### 4. 数学建模国赛：从读题到成稿的 72 小时协作流程

| 阶段      | 核心任务             | Chat                      | Work                   | Codex                  |
|---------|------------------|---------------------------|------------------------|------------------------|
| 0-3 h   | 读题、选题、拆题         | 比较 A/B/C 题；判断数学本质、难点和模型候选 | 整理选题评价表与任务清单           | 先不急著写复杂代码              |
| 3-8 h   | 路线设计、EDA         | 确定每问目标、模型假设、评价指标          | 形成项目执行表：输入/方法/输出/图表/验证 | 读取数据、EDA、缺失/异常/相关性分析   |
| 8-24 h  | 建立 baseline 与主模型 | 讨论模型适配性；解释 EDA            | 同步记录模型路线与结果材料          | 实现模型、统一数据划分、跑基线与对照     |
| 24-40 h | 改进、调参、优化         | 判断改进是否有数学意义               | 整理版本差异、保留可用结论          | 参数搜索、优化算法、批量实验         |
| 40-52 h | 验证与鲁棒性           | 解释误差、敏感性、局限性              | 汇总验证证据                 | 交叉验证、噪声扰动、敏感性分析、复现实验   |
| 52-66 h | 论文写作             | 解释“为什么这样建模、结果意味着什么”       | 整合摘要、模型、图表、结果为论文初稿     | 生成图表、LaTeX 表格、检查编译     |
| 66-72 h | 质检与提交            | 模拟评委审稿，查逻辑漏洞              | 统一术语、图表、章节和支撑材料        | 代码审计：数据泄漏、单位、随机种子、可复现性 |

## 5. 一个完整协作示例：共享单车需求预测与调度

### Step 1 | Chat：先确定“做什么、为什么”

- 将赛题拆成：Q1 数据规律 → Q2 需求预测 → Q3 风险评价 → Q4 调度优化。
- 讨论候选方案后，人来确定：K-means / XGBoost / TOPSIS / MILP（示例）。

### Step 2 | Work：把路线变成项目计划

- 为每一问记录：输入数据、模型、输出指标、所需图表、验证方法、论文对应章节。

### Step 3 | Codex：真正执行

- 建立 data/src/figures/results/paper 目录，完成 EDA、特征工程、模型比较、调试与结果导出。
- 例如统一比较 Linear / RF / XGBoost 的 RMSE、MAE、 $R^2$ ，并固定随机种子避免不可复现。

### Step 4 | 回到 Chat：解释结果

- 不是只写“XGBoost 指标最好”，而是解释：非线性、特征交互、数据规模与模型假设为何使其更适合。

### Step 5 | Codex：补强论文证据

- 继续做交叉验证、敏感性分析、噪声扰动、优化求解，并生成论文级图表。

### Step 6 | Work：整合成稿

- 把题目、方法、公式说明、图、表、实验结果和结论组织成完整论文初稿。

### Step 7 | Chat + Codex：双重质检

- Chat 以评委视角找逻辑漏洞；Codex 从代码侧检查数据泄漏、单位错误、随机性、图表一致性与复现问题。

#### 最重要的比赛原则

让 AI 扩大执行能力，不要让 AI 替代建模主体性。推荐模式：人定模型与验证逻辑 → Codex 公平执行 → Chat 解释 → Work 整理。

## 6. 推荐的数模工程结构

```
CUMCM2026/
├── data/           原始与清洗数据
├── src/            各问代码、预处理、敏感性分析
├── figures/         论文图片
├── results/         CSV / 指标 / 中间结果
├── tables/          自动生成的 LaTeX 表格
├── paper/           main.tex 等论文文件
├── README.md        运行方法与模型说明
└── requirements.txt
```

## 7. 如何用 Plus 为大学科研做准备

目标不是“更会问 ChatGPT”，而是提前练出一条科研 workflow：

### 科研闭环

文献 → 问题 → 假设 → 方法 → 代码 → 实验 → 数据 → 图表 → 解释 → 写作 → 复现

### ① 文献阅读

Chat 拆论文：研究问题、贡献、方法、实验、局限；Deep Research 用于建立一个领域的文献地图。

### ② 方法学习

不要只问定义，要追问：模型假设是什么？何时失效？有什么替代方法？如何设计实验验证？

### ③ 科研代码

用 Codex 从“写一个函数”升级到“维护一个科研代码库”：参数扫描、批处理、测试、复现。

### ④ 实验设计

从“电路/程序跑通”升级到“证明它有效”：变量控制、对照组、不同工况、误差与重复实验。

### ⑤ 项目管理

用 Work 维护研究进度：已完成、未解决、关键风险、下周计划、可写入论文的结果。

### ⑥ 科研写作

LaTeX 与代码联动：程序生成 results.csv / figure.pdf / table.tex，论文自动引用，减少手工抄数。

⑦ 知识沉淀

建立 GitHub Research-Notebook: Literature / Methods / Code / Experiments / Writing / Ideas / Projects。

8. 大学阶段的建议节奏

| 阶段        | Plus 重点用法     | 目标能力                |
|-----------|---------------|---------------------|
| 大一 / 大二前期 | Chat + 文件阅读   | 基础课、编程、读懂技术资料与论文    |
| 大二        | Chat + Codex  | 模型实现、科研代码、实验分析      |
| 大二 / 大三   | Deep Research | 文献综述、领域地图、研究问题意识    |
| 大三        | Codex + Work  | 独立项目、数据分析、完整实验链     |
| 大三 / 大四   | 全流程组合         | 竞赛、大创、毕设、科研论文、可复现项目 |

9. 最容易踩的 7 个坑

- 把 AI 的模型推荐直接当作建模结论，而没有自己检查假设与题意。
- 为了“高级”堆很多模型，却没有 baseline、对照和解释。
- 让 Codex 自己寻找“效果最好模型”，导致评价不公平甚至数据泄漏。
- 只关注  $R^2$  / Accuracy 等指标，不解释结果对应题目中的实际含义。
- 论文里的数字、图表与代码输出不同步，人工复制时引入错误。
- 把 Plus / Pro 当成 API 套餐；实际上 API 单独计费。
- 太早升级 Pro，却没有建立高强度 Work/Codex 工作流，花钱却没有获得对应收益。

10. 最适合你当前阶段的方案

**平时：Plus + 工作流训练**

优先把 Chat、Work、Codex、Deep Research 真正串起来。用历年国赛题和现有工科项目练习“问题 → 代码 → 实验 → 论文”的完整闭环。

**比赛 / 科研冲刺期：再判断 Pro**

如果出现连续高强度 Codex / Work、频繁触发计划限额、多个长任务并行等真实瓶颈，再考虑临时升级 Pro。

### 长期目标

即使没有 AI，你也知道科研下一步该做什么；有 AI 时，你能把执行效率、复现能力和资料组织能力显著放大。

## 11. 三句复盘口诀

**Chat：做什么？为什么？合理吗？**

**Codex：怎么实现、运行、验证和复现？**

**Work：怎么把所有材料组织成最终成果？**

## 参考与说明

产品功能与计划信息可能随 OpenAI 更新而变化。以下官方页面用于校验本 PDF 中的产品描述：

[Plus](#) | [Pro](#) | [Work / Codex](#) | [Agentic credits](#) | [ChatGPT / API billing](#)